

# Partition-Key Design and Capacity Mode in Amazon DynamoDB: An Empirical Performance-Cost Evaluation under Serverless Workloads

Rasool Basha Durbesula | Student ID: 24205478

MSc Cloud Computing — National College of Ireland — Short Explanation Guide

Baseline: Panteli et al. (2026) Future Internet — DOI 10.3390/fi18010053

---

Read aloud in simple English. Numbers come from STATUS.md / results — do not invent live AWS metrics.

## 1. What is the project?

This project measures how Amazon DynamoDB table configuration — partition-key design × capacity mode — changes latency, throttling, consumed capacity, and list-price cost when AWS Lambda drives serverless workloads.

Key designs: K1 simple (orderId); K2 composite (customerId + orderTs); K3 write sharding (orderId#shard, N=10). Capacity: on-demand or provisioned. Live: 12 key cells (K1–K3 × modes × W3/W4), 100k seed, eu-west-1, throttle\_rate 0.0; stack destroyed.

Pitch: “I turn two DynamoDB knobs — partition key and capacity mode — under serverless Lambda load, and meter latency, throttles, capacity, and cost.”

## 2. Problem statement + real-life example

DynamoDB performance is not one number. A hot partition key can throttle even with spare table capacity. On-demand and provisioned price differently. Wrong key or mode wastes money or adds tail latency under burst.

Real-life example — warehouse aisles. K1 = unique barcode per box (spreads well). K2 = aisle = customer (celebrity jam). K3 = split hot aisle into 10 sub-aisles. Capacity mode = rent fixed forklifts (provisioned) vs pay per trip (on-demand).

Prior SQL/NoSQL engine benches give workload language but not RCU/WCU, throttle admission, or per-op price on live metered DynamoDB.

### 3. Baseline paper + brief SQL vs NoSQL

Panteli et al. (2026), Future Internet 18(1):53 (DOI 10.3390/fi18010053). Benchmark SQL vs NoSQL persistence under read-heavy, write-heavy, mixed workloads on a self-hosted single node; rank engines on latency and throughput.

Gap: no RCU/WCU, no throttling admission control, no per-operation price. We keep their workload language and classic metrics; we add metered DynamoDB quantities. Not a container SQL/NoSQL re-bench.

SQL vs NoSQL (short): SQL = fixed tables and joins; strong queries, vertical write scaling. DynamoDB NoSQL = items by key, horizontal partitions; design keys for access patterns, not ad-hoc joins. This thesis is DynamoDB-native configuration under serverless load.

### 4. How we solve it / research question

RQ: How does DynamoDB table configuration (partition-key design × capacity mode) determine the performance–cost trade-off under serverless workloads?

Solution: factorial K1/K2/K3 × on-demand/provisioned × workloads; Lambda generators; measure p99, throughput, throttle\_rate, cost/10k ops (list price × RCU/WCU). Live floor = W3/W4 12/12 cells.

Soft beyond-CA2: W1/W2 live; confirmatory live ANOVA (n=1 saturates interaction). K4 adaptive sharding = code only, off factorial — never say K4 won.

STATUS examples: K1 on-demand W3 p99 ≈ 7.0 ms at ~200 ops/s; K3 often higher p99/cost on W3. Round list-price sum ≈ \$0.251. Must NOT claim Cost Explorer validation or confirmatory ANOVA.

## 5. Dataset / data

Fully synthetic. Terraform tags: data=synthetic. No PII.

- Design catalogue:  $\approx 1,000,000$  orders / 10,000 customers; Zipfian ( $\approx 90\%$  ops  $\rightarrow \approx 10\%$  keys); item sizes 1/8/32 KB.
- Live key-cell seed: 100,000 items (not full 1M); unique prefixes per final; eu-west-1.
- Measured live: W3 mixed  $\sim 200$  ops/s; W4 burst  $200 \leftrightarrow 1000$  ops/s. W1/W2 unfilled live.
- Cost: list-price YAML (config/prices.yaml) — not Cost Explorer-validated.

## 6. Implementation + technologies / AWS services

- Amazon DynamoDB — six configs (K1–K3  $\times$  on-demand/provisioned).
- AWS Lambda — GetItem/PutItem workload generators.
- Terraform (iac/) — tables, Lambda, IAM, CloudWatch, budgets.
- Python Zipfian generator + cost model.
- CloudWatch — consumed capacity metrics.
- S3 — optional result landing.
- pytest + moto — plumbing only (not AWS latency).
- Analysis — keycell summary; exploratory  $n=1$  OLS/KW only.

## 7. End-to-end flow

- Terraform apply → tables + Lambda in eu-west-1.
- Seed 100k synthetic orders for live round.
- Pick cell:  $K1/K2/K3 \times \text{capacity} \times W3/W4$ .
- Warm Lambdas; measure p99, thr, throttle, RCU/WCU.
- Compute list-price cost per 10k ops.
- Write batches.csv / keycell\_summary.csv for all 12 cells.
- Terraform destroy (32 resources); no ddbpk\* leftovers.

## 8. Exact speaking script (dataset + implementation + technologies)

*“My dataset is synthetic orders — planned one million orders and ten thousand customers with Zipfian access and one-, eight-, and thirty-two-kilobyte items. The live key-cell round seeded one hundred thousand items. No real customer data.”*

*“I implemented three partition-key designs and two capacity modes as DynamoDB tables driven by Lambda workload generators, all in Terraform. I measure latency, throttles, consumed capacity, and list-price cost. Technologies are Python, Terraform, DynamoDB, and Lambda. I did not re-run Panteli’s SQL versus NoSQL container bench — I meter DynamoDB configuration under serverless load. After twelve live cells I destroyed the stack.”*

## 9. STATUS snapshot + five Q&A

|                   |                                                      |
|-------------------|------------------------------------------------------|
| <b>Live cells</b> | 12/12 W3/W4 key cells; throttle_rate=0.0             |
| <b>Seed</b>       | 100k live (design target 1M)                         |
| <b>Cost proxy</b> | list-price sum $\approx$ \$0.251 (not Cost Explorer) |
| <b>Destroy</b>    | 32 TF resources destroyed                            |

**Q:** Re-benchmark SQL vs NoSQL like Panteli■?

**A:** No — they ranked engines on one node; I meter DynamoDB key  $\times$  capacity on AWS.

**Q:** Is live data one million rows?

**A:** Design target 1M; live key cells used 100k seed — say honestly.

**Q:** What are K1, K2, K3?

**A:** K1=orderId; K2=customerId+orderTs; K3=orderId#shard (N=10).

**Q:** Any throttling live?

**A:** throttle\_rate=0.0 on all 12 measured W3/W4 cells.

**Q:** Confirmatory ANOVA on live?

**A:** No — n=1 exploratory only; ANOVA is beyond-CA2 soft.